1. Operatii cu Fisiere si Directoare

1a. Creare ierarhie de directoare

Cel mai rapid mod — un singur **mkdir -p**:

```
mkdir -p AcadNet/{calc/{9-10,11-12},is/{9-10,11-12},rl/{9-10,11-12}}

# Verificare structura:
tree AcadNet
# sau
find AcadNet -type d | sort
```

1b. Creare 50 fisiere cu dimensiune fixa

```
# Cu dd (cea mai comuna metoda):
for i in $(seq 1 50); do
dd if=/dev/urandom of=AcadNet/calc/9-10/subiect_$i bs=1M count=12 status=none
done

# One-liner echivalent:
for i in $(seq 1 50); do dd if=/dev/zero of=AcadNet/calc/9-10/subiect_$i bs=1M count=12 status=none;
done

# Cu truncate (mai rapid, fisiere sparse):
for i in $(seq 1 50); do truncate -s 12M AcadNet/calc/9-10/subiect_$i; done

# Verificare dimensiune:
ls -lh AcadNet/calc/9-10/ | head -5
du -sh AcadNet/calc/9-10/subiect_1
```

■ ***TIP:*** *dd if=/dev/urandom = date random reale | if=/dev/zero = mai rapid, date goale*

1c. Afisare dimensiune totala + structura ordonata

```
# Dimensiunea totala a ierarhiei:
du -sh AcadNet/
du -h --summarize AcadNet/

# Structura ordonata 1-50 (tree sorteaza implicit alfabetic, nu numeric!):
tree AcadNet

# Daca tree nu sorteaza corect numeric:
find AcadNet -print | sort -V

# Lista fisiere ordonate numeric:
ls -v AcadNet/calc/9-10/
```

■ ***TIP:*** *Optiunea -V la sort face sortare 'versiune' (numerica naturala): subiect_2 < subiect_10*

1d. Arhiva zip cu toata ierarhia + afisare continut

```
# Creare arhiva zip:
zip -r ierarhie.zip AcadNet/

# Afisare continut fara dezarhivare:
unzip -l ierarhie.zip

# Alternativ cu zipinfo:
zipinfo ierarhie.zip

# Arhiva tar.gz (daca se cere tar):
tar -czvf ierarhie.tar.gz AcadNet/
tar -tzvf ierarhie.tar.gz # afisare continut
```

2. Gestionarea Proceselor si Planificarea Comenzilor

2a. Top 5 procese dupa memorie + renice

```
# Identifica top 5 procese dupa %MEM ale userului curent:
ps aux --sort=-%mem | grep ^$(whoami) | head -5

# Extrage doar PID-urile:
PIDS=$(ps aux --sort=-%mem | grep ^$(whoami) | head -5 | awk '{print $2}')
echo $PIDS

# Renice la 15 pentru fiecare PID:
for pid in $PIDS; do renice -n 15 -p $pid; done

# One-liner complet:
ps aux --sort=-%mem | grep ^$(whoami) | head -5 | awk '{print $2}' | xargs -I{} renice -n 15 -p {}

# Verificare:
ps aux --sort=-%mem | grep ^$(whoami) | head -5
```

■ ***TIP:*** *renice -n 15 = scade prioritatea (range: -20 la 19, 19=cel mai mic prioritate). Necesita sudo pentru valori negative.*

2b. Planificare comanda cu at (ora exacta)

```
# Planificare la 00:00 pe 1 august 2025:
echo 'echo "Hello from AcadNet 2025"' | at midnight August 1 2025
# sau:
echo 'echo "Hello from AcadNet 2025"' | at 00:00 08/01/2025
# sau:
at 00:00 August 1 2025 <<< 'echo "Hello from AcadNet 2025"'

# Listeaza job-urile at programate:
atq

# Sterge un job at (ex: job 1):
atrm 1

# Cu cron (crontab -e):
0 0 1 8 * echo "Hello from AcadNet 2025" >> /tmp/acadnet.log
```

■ ***TIP:*** *Daca 'at' nu e instalat: sudo apt install at && sudo systemctl start atd*

2c. 10 procese sleep + SIGKILL (one-liner)

```
# Porneste 10 sleep-uri in background si trimite SIGKILL:
for i in $(seq 1 10); do sleep 30 & done; sleep 0.5; pkill -SIGKILL sleep

# Varianta cu kill explicit pe PID-uri:
PIDS=(); for i in $(seq 1 10); do sleep 30 & PIDS+=($!); done; kill -9 ${PIDS[@]}

# Varianta cu jobs:
for i in $(seq 1 10); do sleep 30 & done; kill -KILL $(jobs -p)

# Verificare ca au murit:
jobs
```

2d. Proces sleep cu nume personalizat

```
# Creare sleep cu nume specific (folosind exec -a):
(exec -a "de ce rinocerul are corn?" sleep 1000) &

# Verificare:
ps aux | grep "rinocerul"
pgrep -a "rinocerul"

# Alternativ cu script wrapper:
nohup sleep 1000 & # daca nu se cere nume specific
```

3. Personalizarea Terminalului (PS1, cowsay, cursor)

3a. Colorare PS1: user=rosu bold, @=verde bold, host=galben bold

Coduri ANSI: \[\033[COD m\] unde 1=bold, 31=rosu, 32=verde, 33=galben, 0=reset

```
# Adauga in ~/.bashrc:
PS1='\[\033[1;31m\]\u\[\033[1;32m\]@\[\033[1;33m\]\h\[\033[0m\]:\w\$ '

# Aplica imediat:
source ~/.bashrc
# sau:
export PS1='\[\033[1;31m\]\u\[\033[1;32m\]@\[\033[1;33m\]\h\[\033[0m\]:\w\$ '

# Coduri culori utile:
# 30=negru 31=rosu 32=verde 33=galben 34=albastru 35=magenta 36=cyan 37=alb
# 1=bold 0=reset 4=underline
```

3b. cowsay cu Hello Kitty

```
# Listeaza personajele disponibile:
cowsay -l

# Hello Kitty (daca e disponibila):
cowsay -f hellokitty "ACADNET 2025"

# Daca nu e disponibila, cauta in /usr/share/cowsay/cows/:
ls /usr/share/cowsay/cows/ | grep -i kitty

# Instaleaza daca lipseste:
sudo apt install cowsay -y

# Cu figlet combinat (bonus):
figlet "ACADNET 2025" | cowsay -n
```

3c. PS1 cu IP in loc de hostname

```
# Obtine IP-ul (fara masca):
hostname -I | awk '{print $1}'
ip addr show | grep 'inet ' | grep -v '127.0.0.1' | awk '{print $2}' | cut -d/ -f1

# Seteaza PS1 cu IP dinamic:
PS1='\[\033[1;31m\]\u\[\033[1;32m\]@\[\033[1;33m\]$(hostname -I | cut -d' ' -f1)\[\033[0m\]:\w\$ '

# Versiune mai simpla:
PS1="\u@$(hostname -I | cut -d\" \" -f1):\w\$ "

# Adauga in ~/.bashrc si aplica:
source ~/.bashrc
```

3d. Cursor subliniat clipitor

```
# Secvente escape pentru cursor:
# \033[0 q = cursor default (bloc clipitor)
# \033[1 q = bloc clipitor
# \033[2 q = bloc solid
# \033[3 q = subliniat clipitor <-- ACESTA
# \033[4 q = subliniat solid
# \033[5 q = bara verticala clipitoare
# \033[6 q = bara verticala solida

# Seteaza cursor subliniat clipitor:
echo -e "\033[3 q"

# Permanent in ~/.bashrc:
echo -e "\033[3 q" # adauga aceasta linie in ~/.bashrc

# In terminale VTE/Gnome:
printf "\e[3 q"
```

4. Utilizarea Makefile

4a. Regula build — compilare directa

```
# Makefile:
CC = gcc
TARGET = acadnet2025

build:
$(CC) -o $(TARGET) main.c firststep.c
./$(TARGET)

# IMPORTANT: folositi TAB (nu spatii) la indentare!
# Compilare si rulare:
make build
```

4b. Compilare cu .o + reguli run si clean

```
CC = gcc
TARGET = acadnet2025

build:
$(CC) -c zeus.c -o zeus.o
$(CC) -o $(TARGET) zeus.o secondstep.o

run:
./$(TARGET)

clean:
rm -f $(TARGET) *.o
# Atentie: NU sterge zeus.o! Adauga exceptie:
# rm -f $(TARGET) $(filter-out zeus.o, $(wildcard *.o))

# Daca zeus.o exista deja si secondstep.o exista:
build:
$(CC) -o $(TARGET) zeus.o secondstep.o

# Rulare:
make build && make run
```

■ ***TIP:*** *clean trebuie sa stearga executabilele si .o-urile create, DAR NU zeus.o*

4c. Regula pack — arhiva tar.gz cu Makefile si .c/.h

```
CC = gcc
TARGET = acadnet2025

pack:
tar -czvf acadnet_rules.tar.gz Makefile $(wildcard *.c) $(wildcard *.h)

# Alternativ explicit:
pack:
tar -czvf acadnet_rules.tar.gz Makefile *.c *.h 2>/dev/null || \
tar -czvf acadnet_rules.tar.gz Makefile *.c 2>/dev/null || \
tar -czvf acadnet_rules.tar.gz Makefile

# IMPORTANT: regula 'pack' nu trebuie sa fie .PHONY cu dependente care sterg pack
# Adauga .PHONY pentru reguli care nu produc fisiere:
.PHONY: build run clean pack

# Verifica arhiva:
tar -tzvf acadnet_rules.tar.gz
```

5. Gestionarea Permisiunilor

5a. Creare structura database cu fisiere random de 10MB

```
# Creare directoare:
mkdir -p database/{public,secret}

# Creare fisiere cu date random de 10MB:
dd if=/dev/urandom of=database/public/info bs=1M count=10
dd if=/dev/urandom of=database/secret/info bs=1M count=10

# Verificare:
ls -lh database/public/info database/secret/info
```

5b. Arhiva tar.gz + afisare continut fara dezarhivare

```
# Creare arhiva:
tar -czvf database.tar.gz database/

# Afisare continut fara dezarhivare:
tar -tzvf database.tar.gz

# sau:
tar -tf database.tar.gz
```

5c. Setare permisiuni directoare si fisiere

Tabela permisiuni rapida:

Numeric	rwx	Semnificatie
7	rwx	citire + scriere + executare
6	rw-	citire + scriere
5	r-x	citire + executare
4	r--	doar citire
0	---	nicio permisiune

```
# public: acces complet pentru toti (rwxrwxrwx = 777):
chmod 777 database/public
chmod 777 database/public/info

# secret: acces doar owner + grup (rwxrwx--- = 770):
chmod 770 database/secret
chmod 770 database/secret/info

# config: acces doar owner (rw------- = 600):
touch database/secret/config
chmod 600 database/secret/config

# Verificare permisiuni:
ls -la database/
ls -la database/secret/
ls -la database/public/
```

5d. Creare grup dbadmin, user dbowner, user dbuser

```
# Creare grup:
sudo groupadd dbadmin

# Creare utilizatori:
sudo useradd -m -G dbadmin dbowner # dbowner in grupul dbadmin
sudo useradd -m dbuser # dbuser fara access la secret

# Atribuire director secret catre grupul dbadmin:
sudo chown -R dbowner:dbadmin database/secret
sudo chown -R dbowner:dbadmin database/secret/config

# Verificare:
id dbowner
id dbuser
ls -la database/

# Setare parola (daca e nevoie):
sudo passwd dbowner
sudo passwd dbuser
```

6. Analiza Timpilor de Bootare cu systemd

6a. Afisare timpi de bootare

```
# Timp total de bootare:
systemd-analyze

# Detalii pe faze:
systemd-analyze time

# Grafic blame (servicii ordonate descrescator dupa timp):
systemd-analyze blame
```

6b. Servicii ordonate crescator dupa timp

```
# blame listeaza descrescator, deci inversam cu tail+sort sau sort direct:
systemd-analyze blame | sort -k1 -h

# Alternativ (sort numeric pe prima coloana in ms/s):
systemd-analyze blame | tac

# Cel mai elegant - sort by time ascending:
systemd-analyze blame | awk '{print $1, $2}' | sort -h
```

6c. Doar numele serviciilor, fara duplicate de timp

```
# Afiseaza blame ordonat crescator, elimina duplicate de timp, pastrand doar numele:
systemd-analyze blame | sort -k1 -h | awk '!seen[$1]++ {print $2}'

# Varianta detaliata:
systemd-analyze blame | sort -h | awk '{time=$1; name=$2} !seen[time]++ {print name}'

# Daca cerinta e ordonate crescator si unice:
systemd-analyze blame | tac | awk '{print $1, $2}' | sort -k1 -h | awk '!a[$1]++{print $2}'
```

7. Folosirea Git

7a. Configurare nume si email

```
git config --global user.name 'student'
git config --global user.email 'student@acadnet.ro'

# Verificare:
git config --global --list
git config user.name
```

7b. Init repo + copiere passwd

```
# Creare si initializare director:
mkdir acadnet2025
cd acadnet2025
git init

# Gaseste si copiaza fisierul passwd (de obicei e /etc/passwd):
find / -name 'passwd' -not -path '*/proc/*' 2>/dev/null
cp /etc/passwd .

# Adauga si primul commit:
git add passwd
git commit -m "Add passwd file"
```

7c. Branch networking + ifconfig in fisier + commit

```
# Creare si switch pe branch-ul networking:
git checkout -b networking
# sau:
git branch networking && git checkout networking

# Salveaza output ifconfig intr-un fisier 'ports':
ifconfig > ports
# Daca ifconfig nu e disponibil:
ip addr > ports

# Commit:
git add ports
git commit -m "Suspicious ports"

# Verificare:
git log --oneline
git branch
```

7d. Modificare mesaj commit anterior

```
# Modifica mesajul ultimului commit:
git commit --amend -m "Not suspicious ports"

# Daca vrei sa pastrezi modificarile (nu schimba continut, doar mesaj):
git commit --amend --no-edit # pastreaza mesajul curent
git commit --amend -m "Not suspicious ports" # schimba mesajul

# Verificare:
git log --oneline
git log -1 --pretty=%B
```

8. Provocarea Batman vs. Joker (id afiseaza root)

Cerinta: orice user cand ruleaza **id** sa vada informatiile root. Fara alias, fara modificare /usr/bin/id, fara scripturi evidente, reversibil.

Solutia: wrapper in PATH cu prioritate

```
# Pasul 1: Creeaza un director pentru binare custom:
mkdir -p /usr/local/bin/custom

# Pasul 2: Creeaza un wrapper script 'id' in acel director:
cat > /usr/local/bin/custom/id << 'EOF'
#!/bin/bash
/usr/bin/id root
EOF
chmod +x /usr/local/bin/custom/id

# Pasul 3: Pune directorul custom la INCEPUTUL PATH-ului global:
# In /etc/environment sau /etc/profile.d/custom.sh:
echo 'export PATH=/usr/local/bin/custom:$PATH' | sudo tee /etc/profile.d/joker.sh
chmod +x /etc/profile.d/joker.sh

# Pasul 4: Aplica (sau logout/login):
source /etc/profile.d/joker.sh

# Testare:
id # ar trebui sa arate uid=0(root) gid=0(root)...

# REVERSIBIL: stergere fisier si repornire sesiune
sudo rm /etc/profile.d/joker.sh /usr/local/bin/custom/id
```

Solutia alternativa: LD_PRELOAD (avansat)

```
# Alternativa prin LD_PRELOAD - intercepteaza getuid/getgid la nivel de biblioteca
# Creeaza fisier C care suprascrie getuid/getgid:
cat > /tmp/fakeid.c << 'EOF'
#include <sys/types.h>
uid_t getuid(void) { return 0; }
uid_t geteuid(void) { return 0; }
gid_t getgid(void) { return 0; }
gid_t getegid(void) { return 0; }
EOF

gcc -shared -fPIC -o /usr/local/lib/fakeid.so /tmp/fakeid.c

# Seteaza LD_PRELOAD global:
echo 'export LD_PRELOAD=/usr/local/lib/fakeid.so' >> /etc/environment

# REVERSIBIL: sterge linia din /etc/environment
```

■ ***TIP:*** *Solutia cu PATH wrapper e mai simpla si mai robusta. LD_PRELOAD poate fi ignorat de binare SUID.*

BONUS: Comenzi Esentiale de Stiut

Navigare si fisiere

```
pwd # director curent
ls -lah # lista detaliata cu hidden si human-readable
ls -v # sortare numerica naturala
find . -name '*.c' # cauta fisiere .c recursiv
find . -size +10M # fisiere mai mari de 10MB
du -sh * # dimensiune fiecare element
df -h # spatiu disk disponibil
wc -l fisier # numar linii
tail -f /var/log/syslog # monitorizare log in timp real
```

Procese

```
ps aux # toate procesele
ps aux --sort=-%cpu # ordonate dupa CPU
ps aux --sort=-%mem # ordonate dupa memorie
top / htop # monitor interactiv
kill -9 PID # SIGKILL fortat
kill -SIGTERM PID # terminare gradata
pkill -f 'sleep' # kill dupa nume
nohup comanda & # ruleaza independent de terminal
jobs # procese background curente
fg %1 # aduce job 1 in foreground
bg %1 # trimite in background
wait # asteapta toti copiii
```

Permisiuni rapide

```
chmod 777 dir # rwxrwxrwx - toti au totul
chmod 770 dir # rwxrwx--- - owner+grup, altii nimic
chmod 755 dir # rwxr-xr-x - standard pentru directoare
chmod 644 fisier # rw-r--r-- - standard pentru fisiere
chmod 600 fisier # rw------- - doar owner citeste/scrie
chmod -R 755 dir # recursiv
chown user:grup fisier # schimba owner si grup
chown -R user:grup dir # recursiv
ls -la # afiseaza permisiunile
stat fisier # detalii complete permisiuni
```

Git rapid

```
git init # initializeaza repo
git status # starea fisierelor
git add . # adauga tot
git add fisier # adauga specific
git commit -m 'mesaj' # commit
git log --oneline # istoric scurt
git branch # listeaza branch-uri
git checkout -b branch # creeaza si switch
git checkout branch # switch
git merge branch # merge
git diff # diferente nestagiate
git commit --amend -m 'x' # modifica ultimul commit
git reset --hard HEAD~1 # anuleaza ultimul commit
```

Makefile - template complet

```
CC = gcc
CFLAGS = -Wall -g
TARGET = acadnet2025
SRCS = main.c firststep.c
OBJS = $(SRCS:.c=.o)

.PHONY: all build run clean pack

all: build

build: $(OBJS)
$(CC) $(CFLAGS) -o $(TARGET) $(OBJS)

%.o: %.c
$(CC) $(CFLAGS) -c $< -o $@

run: build
./$(TARGET)

clean:
rm -f $(TARGET) $(filter-out zeus.o, $(wildcard *.o))

pack:
tar -czvf acadnet_rules.tar.gz Makefile $(wildcard *.c) $(wildcard *.h)
```